

## Problem Set 1: Tour of Python

Readings: Newman Ch. 2

Due: Wednesday, September 9th at 11:59 PM

**Instructions:** Work through each problem, following the instructions for the requirements of the code and their outputs. Some starting code is provided for each problem on the course website. Problem sets should be submitted in one batch, with each problem uploaded as its own python script (.py file) on the HuskyCT assignment page with the filename: `ps[PROBLEM SET NUMBER]_[QUESTION NUMBER]_[LASTNAME].py`.

**Parameters:** Submitted solutions are restricted to imports of external functions from standard built-in Python libraries (i.e. `math`, `time`, `os`, `sys`, etc.).

---

### 1. THE MADELUNG CONSTANT

In condensed matter physics, the Madelung constant gives the total electric potential felt by an atom in a solid. It depends on the charges on the other atoms nearby and their locations. Consider for instance solid sodium chloride—table salt. The sodium chloride crystal has atoms arranged on a cubic lattice, but with alternating sodium and chlorine atoms, the sodium ones having a single positive charge  $+e$  and the chlorine ones a single negative charge  $-e$ , where  $e$  is the charge on the electron. If we label each position on the lattice by three integer coordinates  $(i, j, k)$ , then the sodium atoms fall at positions where  $i + j + k$  is even, and the chlorine atoms at positions where  $i + j + k$  is odd.

Consider a sodium atom at the origin,  $i = j = k = 0$ , and let us calculate the Madelung constant. If the spacing of atoms on the lattice is  $a$ , then the distance from the origin to the atom at position  $(i, j, k)$  is

$$\sqrt{(ia)^2 + (ja)^2 + (ka)^2} = a\sqrt{i^2 + j^2 + k^2},$$

and the potential at the origin created by such an atom is

$$V(i, j, k) = \pm \frac{e}{4\pi\epsilon_0 a \sqrt{i^2 + j^2 + k^2}},$$

with  $\epsilon_0$  being the permittivity of the vacuum and the sign of the expression depending on whether  $i + j + k$  is even or odd. The total potential felt by the sodium atom is then the sum of this quantity over all other atoms. Let us assume a cubic box around the sodium at the origin, with  $L$  atoms in all directions. Then

$$V_{\text{total}} = \sum_{\substack{i, j, k = -L \\ \text{not } i=j=k=0}}^L V(i, j, k) = \frac{e}{4\pi\epsilon_0 a} M,$$

where  $M$  is the Madelung constant, at least approximately—technically the Madelung constant is the value of  $M$  when  $L \rightarrow \infty$ , but one can get a good approximation just by using a large value of  $L$ . Write a program to calculate the Madelung constant for sodium chloride. The program should:

- Include a user-defined function `madelung(L)` to calculate the Madelung constant for sodium chloride for some input value of  $L$ .
- Use Python's `time` module to time your function for as large a value of  $L$  as you can, while still running in under a minute or so.
- Print the result and the elapsed time.

## 2. THE SEMI-EMPIRICAL MASS FORMULA

In nuclear physics, the semi-empirical mass formula is a formula for calculating the approximate nuclear binding energy  $B$  of an atomic nucleus with atomic number  $Z$  and mass number  $A$ :

$$B = a_1 A - a_2 A^{2/3} - a_3 \frac{Z^2}{A^{1/3}} - a_4 \frac{(A - 2Z)^2}{A} + \frac{a_5}{A^{1/2}},$$

where, in units of millions of electron volts, the constants are  $a_1 = 15.8$ ,  $a_2 = 18.3$ ,  $a_3 = 0.714$ ,  $a_4 = 23.2$ , and

$$a_5 = \begin{cases} 0 & \text{if } A \text{ is odd,} \\ 12.0 & \text{if } A \text{ and } Z \text{ are both even,} \\ -12.0 & \text{if } A \text{ is even and } Z \text{ is odd.} \end{cases}$$

Write a program to calculate the binding energies  $B$  for atoms given some  $Z$  and  $A$ . The program should:

- Include a user-defined function `be_per_nucleon(Z, A)` that takes as its input the values of  $A$  and  $Z$ , and prints out the binding energy per nucleon  $B/A$  for the corresponding atom.
- Use your function to find the binding energy per nucleon of an atom with  $A = 58$  and  $Z = 28$ .
- Given some value of the atomic number  $Z$ , using your function, go through all values of  $A$  from  $A = Z$  to  $A = 3Z$ , to find the one that has the largest binding energy per nucleon. This is the most stable nucleus with the given atomic number. Print the value of  $A$  for this most stable nucleus for  $Z = 28$  and the corresponding value of the binding energy per nucleon.
- Run through all values of  $Z$  from 1 to 100 and print out the most stable value of  $A$  for each one. Determine the value of  $Z$  for which the maximum binding energy per nucleon occurs and print it.

Hint: The total binding energy for this system is about 490 MeV.

The true answer, in real life, is  $Z = 28$  (Nickel).

## 3. BINOMIAL COEFFICIENTS

The binomial coefficient  $\binom{n}{k}$  is an integer equal to

$$\binom{n}{k} = \frac{n!}{k!(n-k)!} = \frac{n \times (n-1) \times (n-2) \times \dots \times (n-k+1)}{1 \times 2 \times \dots \times k}$$

when  $k \geq 1$ , or  $\binom{n}{0} = 1$  when  $k = 0$ . Using this form for the binomial coefficient, write a program that calculates the binomial coefficient. Your program should:

- Include a user-defined function `binomial(n, k)` that calculates the binomial coefficient for given  $n$  and  $k$ . Make sure your function returns the answer in the form of an integer (not a float) and gives the correct value of 1 for the case where  $k = 0$ .

- b. Using your function, print out the first 20 lines of “Pascal’s triangle.” The  $n$ th line of Pascal’s triangle contains  $n + 1$  numbers, which are the coefficients  $\binom{n}{0}$ ,  $\binom{n}{1}$ , and so on up to  $\binom{n}{n}$ . Thus the first few lines are:

```
1 1
1 2 1
1 3 3 1
1 4 6 4 1
```

- c. Calculate and print (i) the total probability that a coin tossed 100 times comes up heads exactly 60 times, and (ii) the probability that it comes up heads 60 or more times. The probability that an unbiased coin, tossed  $n$  times, will come up heads  $k$  times is  $\binom{n}{k}/2^n$ .

#### 4. CATALAN NUMBERS

The Catalan numbers  $C_n$  are a sequence of integers 1, 1, 2, 5, 14, 42, 132... that play an important role in quantum mechanics and the theory of disordered systems. (They were central to Eugene Wigner’s proof of the so-called semicircle law.) They are given by

$$C_0 = 1, \quad C_{n+1} = \frac{4n+2}{n+2} C_n.$$

Write a program that prints in increasing order all Catalan numbers less than or equal to one billion. Your program should:

- Print in increasing order all Catalan numbers less than or equal to one billion.
- Find and print the value of  $n$  for which  $C_n < 1 \times 10^9$ .

#### 5. RECURSION

A useful feature of user-defined functions is **recursion**, the ability of a function to call itself. For example, consider the following definition of the factorial  $n!$  of a positive integer  $n$ :

$$n! = \begin{cases} 1 & \text{if } n = 1, \\ n \times (n-1)! & \text{if } n > 1. \end{cases}$$

This constitutes a complete definition of the factorial which allows us to calculate the value of  $n!$  for any positive integer. We can employ this definition directly to create a Python function for factorials, like this

```
def factorial(n):
    if n==1:
        return 1
    else:
        return n*factorial(n-1)
```

Note how, if  $n$  is not equal to 1, the function calls itself to calculate the factorial of  $n - 1$ . This is called recursion.

Write a program that employs recursion on the following, your program should:

- a. Include a user-defined function `catalan(n)`, that calculates  $C_n$  using recursion. We encountered the Catalan numbers  $C_n$  in Problem 4. With just a little rearrangement, the definition given can be rewritten in the form:

$$C_n = \begin{cases} 1 & \text{if } n = 0, \\ \frac{4n-2}{n+1} C_{n-1} & \text{if } n > 0. \end{cases}$$

- b. Use your function to calculate and print  $C_{100}$ .
- c. Include a Python function `g(m, n)` that employs recursion on Euclid's formula to calculate the greatest common divisor of  $m$  and  $n$ . Euclid showed that the greatest common divisor  $g(m, n)$  of two nonnegative integers  $m$  and  $n$  satisfies:

$$g(m, n) = \begin{cases} m & \text{if } n = 0, \\ g(n, m \bmod n) & \text{if } n > 0. \end{cases}$$

- d. Use this function to calculate and print the greatest common divisor of 108 and 192.